



Done By: Eng. Ahmed Aref Al-Aini

Supervised by: T. Nora Hossain

Task 1: Research Multiple Inheritance in Python and explain it in simple words with a small example.

Multiple inheritances are when a child's class is derived from two or more parent classes. For example, you are inheriting some of the features from your father and some from your mother. In this case, we have to write the names of both parent classes inside the brackets of the child class. The child class will derive the properties from both of the classes. Code Example:

```
2
3 class grandparent:
4     house = 'Three storeyed building'
5     def age(self, number):
6         if number>70:
7             return ('Much Older!')
8         else:
9             return ('Growing towards Old age.')
10
11 class parent:
12     car = 'BMW'
13     def work(self, hours):
14         if hours>40:
15             return ('Very hard working!')
16         else:
17             return ('Average hard working!')
18
19 class child(parent, grandparent):
20     phone = 'One Plus'
21
22 child_object = child()
23 print('My house is a ',child_object.house, ' and I have a ', child_object.car, ' and a ', child_object.phone,' phone.')
24 print('My grandparent is 78 years old. He is ', child_object.age(78))
25 print('My father works for 47 hours a week. He is ', child_object.work(47))
26
```

Output:

```
My house is a Three storeyed building and I have a BMW and a One Plus phone.
My grandparent is 78 years old. He is Much Older!
My father works for 47 hours a week. He is Very hard working!
```

When multiple parent classes have the same method name, Python follows an order called Method Resolution Order (MRO):

MOR says that method or attribute should be executed first when a class inherits from multiple classes. Code Example:

```

inheritance.py > ...
1  class A:
2      def show(self):
3          print("A")
4
5  class B:
6      def show(self):
7          print("B")
8
9  class C(A, B):
10     pass
11
12  c = C()
13  c.show()

```

Output:

```

A

```

Task 2: Research built-in packages and modules in Python and explain them in simple words with simple examples.

Python built-in modules are a set of libraries that come pre-installed with the Python installation. It will become a tedious task to install any required modules in between the process of development that's why Python comes with some most used modules required. These modules provide a wide range of functionalities, from file operations and system-related tasks to mathematical computations and web services.

1- JSON Module in Python

JSON module in Python is used to encode and decode JSON data. JSON format is a widely used on the web to exchange the information. It is extremely useful for reading and writing data in the JSON format

```

import json
data = {
    "name": "Ahmed",
    "age": 23,
    "is_student": True,
    "courses": ["Web Dev", "DS", "DE", "ML"]
}
json_string = json.dumps(data, indent=4)
print(json_string)

```

[1] ✓ 0.0s

```

{
    "name": "Ahmed",
    "age": 23,
    "is_student": true,
    "courses": [
        "Web Dev",
        "DS",
        "DE",
        "ML"
    ]
}

```

Random module:

The Random module in Python is used to generate random numbers and provides the functionality of various random operations such as 'random.randint()', 'random.choice()', 'random.random()', 'random.shuffle()' and many more.

```
> v
import random
num = random.randint(1, 10)
print(f"Random integer between 1 and 10: {num}")
fruits = ["Java", "C", "C++", "Python"]
chosen_fruit = random.choice(fruits)
print(f"Randomly chosen language: {chosen_fruit}")

[2] ✓ 0.0s

.. Random integer between 1 and 10: 6
Randomly chosen language: Java
```

Math Module:

The math module offers mathematical functions used for advanced arithmetic operations. This includes trigonometric functions, logarithmic functions, and constants like pi and e. This module is used to perform complex calculations using Python program.

```
import math
sqrt_val = math.sqrt(64)
pi_const = math.pi
print(sqrt_val)
print(pi_const)

✓ 0.0s

8.0
3.141592653589793
```

Datetime Module:

The Datetime module allows for manipulation and reading of date and time values. Some of the basic methods of "datetime" module are "datetime.date", "datetime.time", "datetime.datetime", and "datetime.timedelta".

```
import datetime
date_today = datetime.date.today()
time_now = datetime.datetime.now().time()
print(date_today)
● print(time_now)

✓ 0.0s

2026-02-21
23:35:12.612397
```

OS Module:

The OS module in Python is used to interact with the operating system and offers OS-level functionalities. For example, interacting with file system, reading directory, and launching application.

```
import os
directory = os.getcwd()
print(directory)
```

✓ 0.0s

d:\الدورة التدريبية\RTS\Data Engineering\A1\practice_python\simple_projects\3_student_management_sytem

Generate + Code + Markdown

Sys Module:

The Sys module in Python provides functions and variables that interact with the Python runtime environment. It allows developers to access Python interpreter attributes and manipulate them. It offers a range of other functionalities, such as input/output stream access, memory info access, and more.

```
import sys
print("Python version:", sys.version)
print("Command line arguments:", sys.argv)
sys.exit(1)
```

⊗ 0.3s Python

Python version: 3.12.3 (tags/v3.12.3:f6650f9, Apr 9 2024, 14:05:25) [MSC v.1938 64 bit (AMD64)]

Command line arguments: ['C:\Users\Ahmed Al-Aini\AppData\Roaming\Python\Python312\site-packages\ipykernel_launcher.py', '--f=c:\Users\Ahmed .

Hashlib Module:

The Hashlib module in Python provides algorithms for message digests or hashing. It allows for data integrity checks using algorithms like SHA-256, MD5, and more.

```
import hashlib
message = "Hello, World!"
hashed = hashlib.sha256(message.encode()).hexdigest()
print(hashed)
```

[7] ✓ 0.0s

.. dfffd6021bb2bd5b0af676290809ec3a53191dd81c7f70a4b28688a362182986f